

第一节

AI 编程的本质、工具、方法论体系

进入 AI 编程具体技术之前，先建立完整认知地图。
讲清楚本质、行业真实状态、工具全景，以及
SDD + Harness 两大方法论的核心理念。
这节课是后面所有学习的认知地基。

认知地基

> @Robert/Oryx Labs



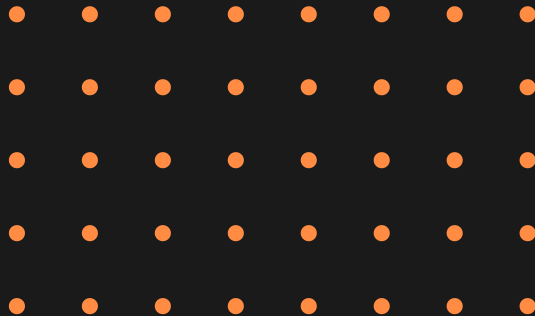
```
bash — 80x24

$ /specify
  ↳ AI 追问边界条件，输出可执行规格

$ /plan
  ↳ 生成技术方案，等你 review 确认

$ /tasks
  ↳ 拆解为独立可验收任务列表

$ /implement
  ↳ 逐任务执行，每步可见可控
```



先照镜子

你上一次用 AI 编程，是下面哪种状态？

A

把需求扔给 AI，反复来回改
最后不确定能不能用

B

学了很多工具和技巧
但换个场景就用不出效果

C

能跑通某类任务
但整体工作流还不成体系

D

有一套工作流，AI 在整条
链路里都在为我高效工作

A / B / C 是当下绝大多数 AI 编程学习者的真实处境。D 是这门课结束后你应该到的位置。本节课建立认知地图，让你知道从哪里到哪里、怎么走。

本节课的目标：建立完整认知地图，让你不再被工具和焦虑裹挟

本节课做三件事

01

讲清本质

程序员的目的没变，变的是路径

路径变化的三层含义，以及它引发的连锁反应。
看清这件事，焦虑消解一半。

02

看清市场和工具

认知三阶段 · 七层工具全景

你卡在哪里。
工具怎么分层。
新工具来了怎么用三个问题快速定位。

03

给出方法论

SDD + Harness Engineering

当下最成熟的两套方法论。
SDD 解决“想”，Harness 解决“做”，合起来是闭环

程序员的杠杆，变了四次

20 年前

底层能力

C / C++ 手写极致性能
代码
操作系统·游戏引擎·数
据库
杠杆：稀缺技术能力

已经历过

10 年前

框架与平台

Java / Python / Rails
快速搭复杂业务系统
杠杆：稀缺平台掌握

已经历过

5 年前

云原生架构

Kubernetes · 微服
务 · 大数据
做出单机时代无法想象
的规模
杠杆：稀缺架构设计

已经历过

今天

驾驭 AI

一个人借助 AI 工具
做出过去一个团队才能
做的产品
这是 AI 编程时代的新
杠杆

新杠杆

每一次淘汰的不是程序员，是没跟上杠杆变化的那批人

AI 编程 = 让 AI 写代码

× 这是最大的误解

程序员这个职业的目的，从来不是写代码

是比别人更快更好地实现某个功能，做别人做不到的事，为用户和业务产生价值

代码是手段

写得多、写得快、写得准
≠ 产出价值
代码量只是过程，不是目的

√ 价值是目的

比别人更快更好地产生价值
= 几十年不变的本质
AI 只是最新的路径变化

路径变了什么：三层递进

	旧路径	新路径
第一层 表层	写代码 手感·代码量·提交数 写得多写得快 = 好工程师	驾驭 AI 写代码 判断·验收·拍板 从手感变成判断
第二层 中层	勤奋 = 产出 加班建立护城河 时间投入线性带来回报	清晰度 = 产出 想不清楚，来回拉扯一天也没成果 勤奋不再是关键变量
第三层 深层	代码生产者 你产出代码，别人关心价值 本分：把代码写好	价值生产者 定义边界·验收产出·对结果负责 过去不属于你，现在全回到你手里

三层变化合起来，是这次 AI 时代和过去任何一次工具迭代最大的不同

路径变化引发的连锁反应

01 杠杆位置上移

这次不同：杠杆本身具备智能。工程师从"使用工具"变成"管理一个能力极强但需要被约束的执行者"。没有前例可以参考

02 责任结构重写

"AI 写错了"不是理由。你承担 AI 产出的整个模块的责任，包括没逐行看过的部分。权力下放，责任也下放了

03 两极分化加速

好工程师驾驭 AI 产出爆炸；不会驾驭的被 AI 拖累，反而更慢。差距从线性变成指数级。站在原地就是退步

04 超级个体可能但门槛更高

AI 把执行力放大了，但判断力和价值感知力 AI 放大了。真正的超级个体 = 判断力 + 执行力 + 价值感知力的合一

个人层面：三个冷静的判断

01 不要把 AI 编程当新职业

过去十年积累的判断、品味、踩坑经验依然是核心资产。表达形式从"我会写"变成"我会让 AI 写，并且我能看出 AI 写错在哪"。不是推倒重来，是升级。

02 重心要从"做"转到"想"

"做"被外包给 AI，"想"全部留给你。想清楚需求比写代码更重要，想清楚架构比记语法更重要。AI 时代的核心训练，是训练"想"的能力。

03 学工具是基础，工具不是目的

真正的成长不是"我会的工具更多了"，是"我能产生的价值更大了"。学工具的人在工具迭代时被淘汰，学价值判断的人反而更强——他能更快上手新工具。

AI 时代工程师真正需要训练的，是"想"的工程化能力

AI 编程认知三阶段：你卡在哪里？

01

把 AI 当写代码工具

Copilot 时代 · 大多数人的起点

生成代码 · 修 bug · 解释源码
天花板：约 1.5x 提速
衡量标准：每天少打多少行代码

入门阶段

02

学工具奇技淫巧

当下最大人群 · 最焦虑

各种 prompt 技巧 · agent 模式 · MCP 集成
越学越乱 · 越乱越焦虑
场景一变就用不出来

大多数人卡在这里

03

意识到这是工作流的事

少数人 · 正在形成共识

AI 进入整条工程链路
从需求到上线全程更高效
共识在形成，但缺方法

训练营带你到这里

越学越焦虑：根源只有一句话

把"用得好 AI"等同于"知道更多 AI 技巧"

手段导向

知道更多工具 = 课越买越多
= 工具越学越多
= 焦虑越来越大
永远有追不完的新技巧
学了就忘，场景一变用不上

√ 结果导向

想清楚要什么 + 用对工具达到
= 不再追新技巧
= 焦虑自然消解
大多数新工具对你暂时不需要
省下注意力，用在更深的地方

真正的尽头不是"学完所有技巧"，是想清楚自己要什么，然后用对工具达到

行业真实状态：4 个判断

01 行业阶段

2026 年仍在基础设施爆发期。GitHub 榜单头部几乎全是基础设施类项目，大规模商业化应用还没出现在头部。类比 1849 年淘金热：大家都在卖镐头铲子

02 工具趋势

Claude Code / Cursor / Codex 核心能力高度趋同，都支持 MCP / Subagents / Spec-Driven 工作流。工具差异化在缩小，工程师之间的差距反而被放大

03 技能投资

学"工具背后的抽象"比追"具体工具"更长青。MCP / SDD / Subagents 这些抽象会比某个具体工具版本活得久得多。工具迭代，抽象不迭代

04 期望管理

IBM + AWS 真实生产数据：开发时间最多减少 30%。营销话术里的 10x 在公开数据里找不到支撑。当前生产形态是"人 + AI 协作"，不是全自动

期望管理：数据会说话

真实生产数据

30%

开发时间减少

来源：IBM + AWS 客户真实生产数据
这已经是顶级水平，不是普遍水平

营销话术

10x

十倍提速

公开真实数据中找不到支撑
连 Anthropic 自家也出过生产事故

按 30% 管理期望，不要按 10x。大多数企业 AI 落地 pilot 没产生显著回报，这是普遍现象

七层工具架构：给每个工具一个位置

L7 方法论层	SDD · Harness Engineering · Vibe Coding
L6 工作流层	Headless Mode · GitHub Actions + AI · CI/CD 自动化
L5 协议与生态层	MCP（全行业共识标准）· gh CLI · 各类 API 集成
L4 协作机制层	Subagents · Worktrees · Plan Mode
L3 配置与约束层	CLAUDE.md · Skills · Permission · Hooks
L2 工具层	Claude Code · Cursor · Codex CLI · Agent SDK
L1 模型层	Claude · GPT · Gemini · DeepSeek · Qwen

看到新工具
先判断
它属于
哪一层

工具数量不是问题。没有分类锚点，才是让你焦虑的根源

新工具来了？三个问题定位它

Q1 它属于哪一层？

七层架构里的什么位置？是新 IDE 插件、新 MCP Server、还是新方法论？先定位，再判断

Q2 它解决什么问题？

在这一层里，和现有工具相比，具体好在哪？如果说不出来具体的，大概率是营销

Q3 你现在需要它吗？

你当前场景里，这一层的需求是什么？现有工具够不够？答案通常是"暂时不需要"—放下，继续做你的事



三问问完，大多数"新工具焦虑"会自动消解。95% 的新工具对你当下不重要

方法论全景：现在有哪些

主轴 SDD

Spec-Driven Development · 规格先行 · Spec-Kit 工具 · 四阶段工作流 · 覆盖完整工程链路

主轴 Harness Engineering

四支柱：System Prompt + Tools + Context + Subagents · 约束 AI 工作边界

底色 Vibe Coding

用熟比会用更重要 · 形成肌肉记忆 · 在项目里默会体验，不需要显性教

远景 Context Engineering

上下文质量决定输出质量 · 方向正确但还在演化期 · 没有标准工具和成熟工作流

补充 ReAct / TAO

Agent 自主规划+执行+反思 · 适合做 Agent 应用场景 · 非通用软件工程主轴

补充 Test-Driven AI

先写测试约束 AI 行为 · SDD 落地的一个具体动作 · 是方法，不是完整方法论

为什么主轴是 SDD + Harness

选方法论的标准只有三条，按这三条过一遍所有方法论：

1 成熟

完整概念体系 + 经过验证的实践，
不是一篇文章的昙花一现

2 可教

有具体可学的步骤、可使用的工具、
可练习的动作，不只是“意会”

3 覆盖完整工程

贯穿从需求到部署的完整软件工程
链路，不只在某个单一环节有效

方法论	成熟	可教	覆盖完整工程
SDD	✓	✓	✓
Harness Engineering	✓	✓	✓
Vibe Coding	✓	△ 意会	△ 只是姿态
Context Engineering	△ 演化中	△ 无标准	△ 概念阶段
ReAct / Test-Driven AI	✓	✓	△ 场景受限

当下同时满足三条的，只有 SDD 和 Harness Engineering

SDD：规格是真理，代码服务于规格

Spec-Driven Development · 把"想清楚"这件事工程化，把模糊需求变成 AI 能消化的清晰输入

1

/specify

把需求想清楚

描述想做什么，AI 追问边界条件和约束，输出可执行的需求规格。模糊想法变成清晰文档。

2

/plan

制定实施方案

AI 基于规格给出技术方案，你 review 确认方向。不满意可以让 AI 重来，方向对了再继续。

3

/tasks

拆成任务列表

方案拆成独立可执行任务，每个任务有明确输入、输出和验收标准。你知道什么叫完成。

4

/implement

逐步执行验收

AI 逐任务执行，你逐任务验收。做完一个确认没问题再做下一个。可见可控可回溯。

没有 SDD，你和 AI 的协作就是凭直觉聊天。有了 SDD，你们之间有了明确契约

Harness Engineering: 给 AI 装上挽具

AI 是匹烈马，Harness 是让它能为你拉车的那套挽具——让 AI 从“放飞的执行者”变成“可控的协作者”

01

System Prompt

CLAUDE.md / AGENTS.md

一上来就建立规则。项目是什么、代码风格、命名约定、目录结构、禁止操作。AI 启动就知道这个项目的规矩。

02

Tools

Skills / Slash Commands / MCP

把可复用的能力封装成工具集。团队约定固化下来，AI 直接调用，不需要每次重新描述。通用 + 项目级两层。

03

Context

Subagents / Memory / Worktrees

让 AI 在合适的上下文里工作。Subagents 任务隔离防污染，Worktrees 并行加速，Plan Mode 先规划再执行。

04

Subagents

Permission / Hooks / Plan Mode

事前约束 + 事后检查。Permission 在事前限制 AI 能做什么，Hooks 在事后自动检查结果，形成安全网。

只学 SDD 不学 Harness: spec 写得好，但 AI 跑偏了没办法管住

SDD + Harness: 一个解决想, 一个解决做

SDD

解决"想"的问题

把"想清楚"工程化。
通过结构化的 Spec 把模糊需求变成 AI 能消化的清晰输入。

Spec 是你和 AI 之间的明确契约。
Spec 写清楚了, AI 执行的稳定性会有数量级的提升。

没有 SDD, AI 靠猜。
有了 SDD, AI 靠规格。



Harness

解决"做"的问题

把"AI 怎么做"工程化。
System Prompt 建立规则,
Tools 封装能力, Context 管理
避免上下文污染, Subagents
任务隔离, Permission 事前
约束, Hooks 事后验证。

AI 从"放飞的执行者"
变成"可控的协作者"。

共同底层假设

**AI 不是被信任的
是被工程化使用的**

两个合起来才是闭环: SDD 告诉 AI 做什么, Harness 保证 AI 做对

其他方法论的位置

主轴定了，不代表其他方法论不重要。它们都在认知地图上，只是角色不同。

底色

Vibe Coding

在三个项目里反复跑同一套工作流，自然就形成了。不需要单独讲，需要体验中默会。跑完三个项目，Vibe 自然就有了。

长期远景

Context Engineering

方向我非常看好。但它目前处于概念演化期，没有标准工具、没有成熟工作流。2-3 年后成熟了，会是下一门课的主轴。

延伸阅读

ReAct / Test-Driven AI / Plan-then-Execute

各自在特定场景有价值。学完 SDD + Harness 之后，你手里有了核心框架，再去看这些，能自己判断什么时候用、用到什么程度。

主轴反复锤炼，景观完整呈现，边界清晰交代——这是本节课对方法论的整体处理

本节总结 + 下节预告

这节课建立了

▶ 本质认知

目的没变，路径变了三层。手感→判断，勤奋→清晰度，代码生产者→价值生产者。

▶ 市场地图

认知三阶段 · 4 个行业判断 · 30% vs 10x · 七层工具架构 · 新工具三问。

▶ 方法论地基

SDD 解决"想"，Harness 解决"做"，合起来是当下最完整的闭环。

第二节预告

▶ AI 时代程序员的核心竞争力

三层竞争力：用熟 AI + 工程纪律 + 复利型资产。

▶ 怎么卷，卷对方向

四个真实建议 · 业务开发 vs AI 开发：全都要，不是选择题。

▶ Oryx 社区

四个项目和长期共建入口 · 开源贡献怎么起步。

带着这张认知地图继续，后面所有具体技术都会有地方可以落